



P r o f e s s i o n a l E x p e r t i s e D i s t i l l e d

Mastering ServiceNow

Conquer ServiceNow by getting to grips with the power behind the platform

Martin Wood

[PACKT] enterprise 
PUBLISHING professional expertise distilled

Mastering ServiceNow

Conquer ServiceNow by getting to grips with the power behind the platform

Martin Wood



BIRMINGHAM - MUMBAI

Mastering ServiceNow

Copyright © 2015 Packt Publishing

All rights reserved. No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, without the prior written permission of the publisher, except in the case of brief quotations embedded in critical articles or reviews.

Every effort has been made in the preparation of this book to ensure the accuracy of the information presented. However, the information contained in this book is sold without warranty, either express or implied. Neither the author(s), nor Packt Publishing, and its dealers and distributors will be held liable for any damages caused or alleged to be caused directly or indirectly by this book.

Packt Publishing has endeavored to provide trademark information about all of the companies and products mentioned in this book by the appropriate use of capitals. However, Packt Publishing cannot guarantee the accuracy of this information.

First published: May 2015

Production reference: 1260515

Published by Packt Publishing Ltd.
Livery Place
35 Livery Street
Birmingham B3 2PB, UK.

ISBN 978-1-78217-421-9

www.packtpub.com

Cover image by Vivek Sinha (vs@viveksinha.com)

Credits

Author

Martin Wood

Project Coordinator

Mary Alex

Reviewers

Jason Butz

Jonathan Jacob

Ruen D Smith

Matt White

Proofreaders

Stephen Copestake

Safis Editing

Indexer

Tejal Daruwale Soni

Commissioning Editor

Edward Gordon

Graphics

Abhinash Sahu

Acquisition Editor

Sam Wood

Production Coordinator

Aparna Bhagat

Content Development Editor

Rohit Kumar Singh

Cover Work

Aparna Bhagat

Technical Editor

Menza Mathew

Copy Editors

Sonia Michelle Cheema

Jasmine Nadar

Aditya Nair

Merilyn Pereira

About the Author

Martin Wood has spent the last few years spreading the word about ServiceNow. His first exposure to the platform was as one of its first customers in the UK in 2008, when he built a custom ITSM application. He enjoyed the experience so much that he joined the company. Now, he works with a variety of clients, ranging from blue-chip enterprises to SMEs, helping them utilize the power of ServiceNow. Martin is passionate about helping people make an informed choice and enjoys speaking at ServiceNow user conferences.

He lives in the beautiful Berkshire countryside with his wife, Sarah. They both love exploring the world and enjoying good food and great wine!

About the Reviewers

Jason Butz began learning about ServiceNow in 2013, and with the aid of several years of experience with web technology, he quickly started pushing the limits of what the framework could do. He has made numerous customizations and integrations to it to accomplish business goals and increase efficiency. He has since moved on to focus on web application development; however, it remains a valuable resource for people involved in the development of ServiceNow. It also acts as a code reviewer to ensure that quality is maintained.

Jonathan Jacob has been honing his expertise in ITSM since 2011. After being introduced to ServiceNow while working for a large publishing company, He joined a ServiceNow Solutions partner. There, he was able to consult for a number of industries from higher education to telecommunications. He has since returned to his roots in publishing as an architect in ServiceNow at The New York Times Company. Outside the world of ITSM, Jonathan is an avid cook and a hamburger enthusiast.

Ruen D Smith has been using ServiceNow since 2011. He has been a part of several client companies of ServiceNow, such as Deutsche Bank, UBS, National Grid, Gatwick Airport, and Microsoft. He has previously worked for ServiceNow partners, such as Innovise, Focus group, and HCL. He works as an administrator and is a Certified Implementation Specialist.

Matt White is a cloud security architect, with over 13 years of experience in information security. He has a reputation for designing and implementing large-scale security solutions for cloud platforms and service providers since 2011. He is currently building ServiceNow's third-party integrations for security products and enabling security automation for ServiceNow's security operations team. In previous roles with EMC and VMware, he served as the chief security architect for several cloud initiatives, including Mozy Online Backup, Cloud Foundry, and the vCloud Air service.

I'd like to thank Martin Wood and Packt Publishing for the opportunity to review and contribute to the publication of this book. It represents a broad coverage of our cloud platform and is a must-have for any administrator or developer creating business solutions with ServiceNow. I am always amazed and excited to see what innovations customers and partners are implementing with this great product.

www.PacktPub.com

Support files, eBooks, discount offers, and more

For support files and downloads related to your book, please visit www.PacktPub.com.

Did you know that Packt offers eBook versions of every book published, with PDF and ePub files available? You can upgrade to the eBook version at www.PacktPub.com and as a print book customer, you are entitled to a discount on the eBook copy. Get in touch with us at service@packtpub.com for more details.

At www.PacktPub.com, you can also read a collection of free technical articles, sign up for a range of free newsletters and receive exclusive discounts and offers on Packt books and eBooks.



<https://www2.packtpub.com/books/subscription/packtlib>

Do you need instant solutions to your IT questions? PacktLib is Packt's online digital book library. Here, you can search, access, and read Packt's entire library of books.

Why subscribe?

- Fully searchable across every book published by Packt
- Copy and paste, print, and bookmark content
- On demand and accessible via a web browser

Free access for Packt account holders

If you have an account with Packt at www.PacktPub.com, you can use this to access PacktLib today and view nine entirely free books. Simply use your login credentials for immediate access.

Instant updates on new Packt books

Get notified! Find out when new books are published by following @PacktEnterprise on Twitter or the *Packt Enterprise* Facebook page.

Disclaimer

Martin wrote this book in his personal capacity. The views expressed are his own, and do not necessarily reflect that of ServiceNow.

Table of Contents

Preface	xvii
Chapter 1: ServiceNow Foundations	1
Diving into the infrastructure	2
Being in charge	2
Changing your instance	2
Choosing functionality with plugins	3
Digging into hosting	4
Knowing the nodes	5
Exploring the database	5
Introducing the Gardiner Hotel data structure	7
Creating tables	9
Adding fields	10
Knowing what's happening	11
Introducing the dictionary	12
The Globally Unique Identifier	13
Building hierarchical tables	14
Benefiting from an object-oriented design	15
Extending the User table	15
Interacting with hierarchical tables	16
Viewing hierarchical tables	18
Overriding field properties	19
Understanding the background behavior	19
Changing class	21
Storing data	22
Attachments	23
Setting properties	24
Reference fields	24
Creating a reference field	25
Using Reference Qualifiers	29

Dot-walking	31
Using derived fields	31
Dynamic creation	32
Deleting records	33
Many-to-many relationships	35
Building a many-to-many table	36
Adding fields to a many-to-many table	38
Deleting a many-to-many table	39
Glide Lists	39
Building the interface	40
Lists	40
Choosing the fields to show	41
Having reference fields on lists	41
The varied capabilities of lists	42
Forms	42
Creating useful forms	43
Adding related and embedded lists	43
Defining your own Related Lists	44
Creating tags and bookmarks	45
Adding a bookmark	45
Defining a tag	46
Enjoying views	46
Controlling views	47
Menus and modules	48
Specifying a view	48
Setting a filter	48
Building the right modules	49
Summary	49
Chapter 2: Server-side Control	51
A brief history of JavaScript	52
Running background scripts	52
Rhino – the JavaScript engine powering ServiceNow	54
Accessing Java	55
Appreciating server-side code	56
Comparing engine speed	57
Using GlideRecord (and its friends)	58
Accessing data from GlideRecord	60
Walking through reference fields	61
Converting data types	61
The surprising results of GlideElement	62
Getting the value another way	63
Dealing with dates	63
Counting records with GlideAggregate	64

Business Rules	65
Predefined variables	65
Setting the table	66
Displaying the right table	66
Conditioning your scripts	67
Having good conditions	68
Controlling the database	69
Controlling database queries with Business Rules	69
Choosing when to execute – before, after, and really after	70
Defaulting data	71
Validating information	73
Working with dates	75
Updating information	76
Running things later with system scheduling	77
Display Business Rules	78
Preventing colliding code	78
Global Business Rules	81
Script Includes	81
Creating classes	82
Coding a class	84
Using classes to store and validate data	85
Extending classes	86
Extending the class example	86
Utility classes	87
Providing utility classes	87
Storing functions	87
Having functions in Script Includes	88
Client Callable Script Includes	88
Special function calls	88
Specifying the view using code	89
Enforcing a Data Policy	89
Forcing a comment using Data Policy	90
Specifying dynamic filters	90
Showing Guests that are 'Me'	91
Scripting Reference Qualifiers	92
Showing only guests with reservations	92
Summary	94
Chapter 3: Client-side Interaction	97
Building a modern interface	98
A microsecond of Human Computer Interaction	98
The power and pitfalls of AJAX	99
Choosing a UI Action	100
Finding the current table	101

Displaying UI Actions at the right time	102
Using the Condition field	103
Running client- or server-side code	103
Saving and redirecting	105
Converting a record from Reservation to Check-in	106
Running the client-side script	107
Proving the basics	107
Meeting GlideRecord again	108
Making synchronous AJAX	109
The defined function	110
The anonymous function	110
Using callbacks effectively	111
Single set of data	111
Avoiding GlideRecord	112
Managing fields with a UI Policy	112
Manipulating the form	112
Client-side conditioning	113
Forcing a comment on reservations	113
Controlling UI Policies	114
Manipulating the form with GlideForm	115
Using GlideForm	115
Choosing a UI Policy	117
Meeting Client Scripts	118
Sending alerts for VIP guests	118
Changing, submitting, loading, and more	120
Validating the contents of fields	121
The disappearance of current	122
Translating client scripts	123
Efficiently transferring data to the client	124
Writing a Script Include for GlideAjax	124
Using GlideAjax	125
Passing data when the form loads	126
Creating a Display Business Rule	126
Using scratchpad on the client	127
Storing data in the session	128
Controlling lists with Context Menus	129
Finding out about the list	130
Opening a new tab	131
Customizing and extending the platform	132
Firing on more events	132
Using built-in libraries	133
What could go wrong	134

Taking control of the browser	135
Data Policy saves the day	135
Summary	136
Chapter 4: Getting Things Done with Tasks	137
Introducing tasks	138
Looking at the Task table	138
The important fields	139
Populating fields automatically	141
Recording Room Maintenance tasks	142
Working with tasks	145
Working without a queue	146
Working socially	147
Organizing groups and users	148
Using Additional comments and Work notes	153
Understanding the State field	154
Breaking down states	155
Configuring different states	155
Navigating between states	157
Creating room maintenance states	158
Enforcing on the server	160
Adding a Reference Qualifier	160
Removing states with Client Scripts	161
Automating an assignment based on state	162
Using Data Lookup	162
Drag-and-drop automation with Graphical Workflow	164
Running a workflow	164
Exploring under the covers	165
Appreciating activities	166
Using data-driven workflows	167
Approving tasks	168
Making the decision	169
Understanding what you are approving	169
Asking for approval for the repair team	170
Performing the approval	171
Starting up the workflow	173
Monitoring progress	174
Using the Service Catalog	175
The different types of Catalog Items	175
Creating a Record Producer	176
Adding more information	177
Routing the submitted request with templates	178
Testing the Record Producer	179

Understanding the data behind the Service Catalog	181
Comparing records and Catalog Items	182
Understanding Request Fulfilment	184
Checking out	185
Using the request tables	187
Scripting variables	187
Service Level Management	187
Exploring the SLA data structure	188
Timing an SLA	188
Travelling through time	189
Enjoying relativity	190
Customizing condition rules	190
Avoiding a breach	191
Working SLAs	192
Ensuring maintenance is quick	192
Summary	197
Chapter 5: Events, Notifications, and Reporting	199
Dealing with events	199
Registering events	200
Firing an event	200
Sending an e-mail for new reservations	201
Scheduling jobs	202
Adding your own jobs	203
Creating events every day	203
Running scripts on events	205
Creating tasks automatically	205
Sending e-mail notifications	207
Setting e-mail properties	207
Tracking sent e-mails in the Activity Log	208
Assigning work	209
Sending an e-mail notification on assignment	209
Sending informational updates	214
Sending a custom e-mail	214
Sending e-mails with Additional comments and Work notes	216
Approving via e-mail	218
Using the Approval table	219
Testing the default approval e-mail	220
Specifying Notification Preferences	220
Subscribing to Email Notifications	221
Creating a new device	222
Sending text messages	223
Delivering an e-mail	224
Knowing who the e-mail is from	226

Receiving e-mails	227
Determining what an inbound e-mail is	227
Creating Inbound Email Actions	228
Accessing the e-mail information	229
Approving e-mails using Inbound Email Actions	230
Updating the Work notes of a Maintenance task	231
Having multiple incoming e-mail addresses	231
Using the Email Accounts plugin	232
Redirecting e-mails	233
Processing multiple email address	233
Recording Metrics	234
The difference between Metrics and SLAs	235
Running Metrics	235
Scripting a Metric Definition	236
Monitoring the duration of Maintenance tasks	236
Flattening data with Database Views	237
Creating a Metric Instance Database View	237
Reporting	239
The functionality of a list	240
Using reports elsewhere	240
Sending a shift handover report	241
Analytics with ServiceNow	242
Basic trending with Line Charts	243
Performance Analytics	244
Making sense of reports	244
Ensuring consistency	245
Using outside tools	246
Building homepages	247
Creating a Maintenance homepage	248
Making global homepages	249
Editing homepages	250
Counting on a homepage	250
Optimizing homepages	252
Summary	252
Chapter 6: Exchanging Data – Import Sets, Web Services, and Other Integrations	255
Beginning the Web Service journey	256
Pulling data out of ServiceNow	256
Downloading file-based data	256
Automatically download data using cURL	257
Using more URL parameters	259
Choosing the fields	259
Specifying the records	260

Pulling data designed for the Web	260
Feeding tasks into an RSS reader	261
Cleaning up with SOAP Direct Web Services	262
Grabbing JSON	267
Using REST	267
Bringing it in using Import Sets	269
Specifying the Data Source	270
Creating an Import Set Table	271
Cleaning up Import Set Tables	272
Dealing with XML files	272
Getting data	274
Transforming the data	276
Creating a Field Map	276
Scripting in Transform Maps	278
Posting data to an Import Set	281
Keeping Import Sets running	281
Importing users and groups with LDAP	282
Importing users from an LDAP server	283
Reviewing the configuration	284
Altering the Transform Maps	285
Building Web Service Import Sets	288
Using a Web Service Import Set WSDL	288
Connecting to Web Services	289
Creating tasks after assignment	290
Testing the Web Service	291
Sending the message	292
Using SOAPMessage effectively	295
Building REST Messages	296
Building custom interfaces	297
Creating Scripted Web Services	297
Doing multiplication with a Scripted Web Service	297
Introducing Processors – the ultimate in flexibility	299
Creating a custom processor interface	299
Building a processor	300
Hosting a custom WSDL	301
Integrating with the ECC Queue	301
Using the ECC Queue	302
Introducing the MID server	303
Picking up jobs	303
Installing the MID server	304
Setting up the server	304
Using the MID server	305
Running a custom command	305

Running JavaScript on the MID server	307
Using MID server Background Scripts	307
Interacting with the ECC Queue	308
Working with parameters and MID server Script Includes	309
Using Java on the MID Server	310
Authenticating and securing Web Services	312
Designing integrations	312
Transferring bulk data	312
Real-time communication	313
Communicating through the firewall	314
Summary	315
Chapter 7: Securing Applications and Data	317
Understanding roles	318
Defining a role	318
Assigning roles to users	319
Differentiating between requesters and fulfillers	320
Using impersonation	321
High Security Settings	322
Elevating security	322
Controlling access to applications and modules	323
Controlling access to modules with groups	323
Protecting data with Contextual Security	326
Understanding Contextual Security	327
Specifying rows and fields to secure	328
Securing rows	328
Controlling fields	330
The order of execution	330
Executing the row and then the field	330
Rules are searched for until one is found	330
Defaults are possible	331
The table hierarchy is understood	331
Multiple rules with the same name are both considered	331
Field rules check the table hierarchy twice	331
Summarizing the execution	332
Scripting and Access Controls	332
Securing other operations	333
Outlining the scenarios	334
Conditioning Contextual Security	334
Editing the automatic security rules	335
Testing using impersonation	336
Setting security rules quickly	337
Scripting a security rule	338
Using security rules effectively	340

Encrypting data	341
Evaluating encryption gateways	342
Introducing Domain Separation	343
Defining a domain	344
Applying Domain Separation	344
Organizing domains	345
Introducing global	346
Understanding domain inheritance	346
Turning on Domain Separation	347
Setting domains	349
Exploring domain visibility	349
Understanding Delegated Administration	350
Overriding configuration	350
Displaying different messages for different domains	351
Creating more domain relationships	353
Using Domain Separation appropriately	354
Authenticating users	355
Using internal authentication	355
Controlling authentication	355
Using an LDAP server for authentication	356
Enabling Single Sign On through SAML	358
Logging out	359
Configuring Single Sign On	359
Navigating to the side door	360
Preventing access to the instance	360
Securing Web Services	361
Using WS-Security	363
Improving security with signatures	363
Mutual authentication	364
Setting up outbound Mutual Authentication	364
Summary	365
Chapter 8: Diagnosing ServiceNow – Knowing What Is Going On	367
Building a methodology	368
Identifying the issue	368
Looking at the System Log	368
Writing to the System Log	369
Using the file log	370
Logging appropriately	371
Using the debugging tools	372
Debugging Business Rules	372
Debugging Contextual Security Rules	373

Enabling the JavaScript Debugger	376
Controlling Business Rules on the fly	376
Seeing client-side messages	378
Logging to the JavaScript Log	379
Watching fields	380
Finding out what set the Assignment group	380
Tracking each page request	382
Recording the time taken	382
Monitoring the instance's performance	383
Recording the browser's perspective	384
Going through other logs	386
Finding slow database transactions	387
Classifying slow queries	388
Examining the Slow Query log	389
Understanding behavior	390
Seeing the plan	390
Dealing with other performance issues	391
Managing large tables	392
Archiving data	393
Rotating and extending through sharding	394
Choosing table extension	395
Selecting table rotation	395
Auditing and versioning	395
Turning on auditing	395
Viewing audit	396
Using auditing responsibly	396
Versioning configuration	397
Reviewing the changes	397
Optimizing hardware resources	398
Controlling resources with semaphores	399
Accessing the system internals	400
Understanding the ServiceNow Performance homepage	400
Flushing the system cache	403
Accessing system stats	404
Summary	406
Chapter 9: Moving Scripts with Clones, Update Sets, and Upgrades	409
Using your instances	410
Serializing records to XML	410
Exporting and importing serialized XML	411
Transporting data via XML	412

Recording configuration in Update Sets	413
Capturing configuration	414
Transferring an Update Set	415
Applying an Update Set	416
Understanding multiple Update Sets	417
Relying upon other updates	418
Managing Update Sets	419
Using the wrong Update Set	420
Working with Workflows	421
Having the wrong IDs	421
Backing out Update Sets	422
Moving away from Update Sets	423
Cloning instances	423
Preserving and excluding data	424
Using clones effectively	425
Packaging with the App Creator	426
Creating applications quickly	426
Exporting an application to an Update Set	429
Versioning records	430
Capturing configuration	430
Synchronizing with Team Development	431
Having a parent	432
Comparing instances	432
Using the Team Dashboard	434
Pulling changes	435
Pushing updates	436
Working with multiple development instances	438
Deciding between Update Sets and Team Development	439
Sharing with Share	439
Adding more with plugins	440
Activating plugins	440
Choosing the right plugin	442
Upgrading ServiceNow	442
Understanding upgrades	443
Configuration and Customization	444
Applying upgrades	446
Reverting customizations and restoring out of the box	447
Upgrading instances automatically	448
Managing instances	448
Summary	449

Chapter 10: Making ServiceNow Beautiful with CMS and Jelly	451
Designing a portal	452
Giving self-service access	452
Understanding the options	452
Choosing a portal	455
Building a portal with CMS	456
Getting the assets	456
Structuring a site hierarchy	457
Designing the Pages	457
Finalizing the mock-ups	458
Configuring the pages	459
Making a div-based layout	460
Setting the Site	461
Creating a Page	461
Including the content	462
Adding some style	466
Copying pages	470
Styling the form	472
Creating a Self-Service form	472
Adding more pages	475
Populating the menus	476
Locking down the data	479
Testing the site	483
Creating truly custom pages	484
Creating a UI Page	485
Adding interactivity to UI Pages	486
Including UI Macros	487
Dynamically creating content with Jelly	488
Touching on XML	489
Looping with Jelly	489
Expanding on Jelly	491
Accessing Jelly and JavaScript variables	492
Mixing variables	492
Using JEXL and escaping	493
Setting Jelly variables	494
Caching Jelly	495
Mixing your phases	497
Using Jelly in the CMS	497
Using Dynamic Blocks	497
Specifying Content Types	498
Creating CMS Lists	499
Building Detail Content	501

Understanding the link structure	502
Improving lists and forms	502
Final testing	505
Including Jelly in the standard interface	507
Adding Formatters	507
Decorating and contributing to fields	509
Launching a dialog box	510
Launching UI Pages with GlideDialogWindow	510
Fitting in a form	511
Displaying any page with GlideBox	512
Summary	513
Chapter 11: Automating Your Data Center	515
Servicing the business	516
Discovering devices	517
Building relationships	517
Using the BSM in ServiceNow	520
Storing relationships and CIs	521
Visualizing the extensions	522
Linking the CIs together	522
Separating relationships and classes	523
Building your own BSM with Discovery	523
Remembering the MID server	524
Performing the Disco steps	526
Finding devices with Shazzam	526
Configuring IP addresses for Shazzam	527
Automatically finding IP addresses	529
Shouting out Shazzam	529
Configuring Shazzam	530
Classifying CIs	530
Sensing the classification	532
Classifying other devices	533
Dealing with credentials	533
Identifying the CI	534
Getting the right information	534
Performing identification	535
Exploring the CI	536
Understanding exploration probes	536
Creating custom probes and sensors	536
Relating CIs together	538
Ensuring high-quality data	538

Associating CIs to Business Services	539
Using APD	540
Summing up Discovery	541
Automating services with Orchestration	543
Automating password resets with Orchestration	544
Running the Automation workflow	545
Running custom commands	546
ServiceNow Cloud Provisioning	547
Introducing virtual machines	547
Requesting a virtual machine with Cloud Provisioning	548
Configuration Automation	549
Controlling servers with Puppet	550
Creating the catalogs	550
Configuring Puppet and ServiceNow	551
Cooking with Chef	551
Folding Chef into ServiceNow	552
Running Configuration Automation	553
Summary	553
Index	555

Preface

Congratulations! You have just become the ServiceNow System Administrator for Gardiner Hotels — one of the most successful chains in the region. The CIO realized that ServiceNow exactly fits his needs; a suite of feature-rich applications built upon a robust platform that makes the process of configuration and customization quick and easy. The SaaS architecture means that installation time is virtually nonexistent, which means that you can concentrate on what you want to do. The single tenancy model gives you power to configure and customize the system as needed, but without worrying about hosting or availability. You are in control of this power!

The simplicity of the basic operation is a little alluring. You can easily log in and start working. However, don't be fooled! Due to the breadth and depth of options, ServiceNow can quickly become overwhelming. Just like any other technology, ServiceNow has its own language, best practices, and idiosyncrasies. This is where this book comes in. By focusing on key areas within the platform, we look beyond the basics, in order to understand exactly what is going on. After reading this book, you should have a much better knowledge of how ServiceNow is best utilized, enabling you to create new custom process applications, and configure and customize built-in functionalities. All the applications you may use are built on top of the ServiceNow platform; by understanding this, you will be able to tackle almost any app.

What is in this book

Each chapter dives deep into the foundations of the platform. The book focuses on the specifics of ServiceNow, not what JavaScript is. We'll look at the hows, but more importantly, the whys. We'll explore options, and there is advice, but always along with justification. By understanding how things work, you will be better equipped to make your own decisions.

In this book, you will be configuring ServiceNow to be more useful to Gardiner Hotels. We'll achieve this by building out a sample Hotel Management application designed to showcase many of the capabilities within ServiceNow.

Chapter 1, ServiceNow Foundations, looks at how ServiceNow is structured from an architectural perspective. We explore how the platform is hosted and then dive into the building block of tables, fields, and building interfaces.

Chapter 2, Server-side Control, shows how you can implement your business logic, and then start to automate, validate, and verify data and processes.

Chapter 3, Client-side Interaction, explores how you can make the life of the people using your application just a little bit better by providing validation, feedback, and quick interaction techniques.

Chapter 4, Getting Things Done with Tasks, the Service Catalog and Service Portfolio looks at some of the base application functionalities in ServiceNow. Understand how a task-driven process system is kickstarted by the Task table, by taking advantage of Graphical Workflow, the Service Catalog and Service Level Management.

Chapter 5, Events, Notifications, and Reporting, introduces another level of interaction with your users, by generating reports and scheduling jobs, and handle incoming and outgoing email. Keep everyone informed about what's happening.

Chapter 6, Exchanging Data – Import Sets, Web Services, and Other Integrations, is about importing data from other systems, integrating ServiceNow in your application landscape. No instance is an island!

Chapter 7, Securing Applications and Data, focuses on the challenges of protecting your most important assets: your information. We make sure the right people have the right data.

Chapter 8, Diagnosing ServiceNow – Knowing What Is Going On, helps you when things go wrong. Troubleshooting and investigation hints and tips are explored, so you can get back to full power quickly.

Chapter 9, Moving Scripts with Clones, Update Sets, and Upgrades, builds on the previous chapters to explore how you can get your hard work in the right place. Understand how upgrades work, and how teams can work together to get stuff done.

Chapter 10, Making ServiceNow Beautiful with CMS and Jelly, focuses on creating a self service portal, but also discusses how the ServiceNow interface is built. Having a good looking, well-designed frontend really enhances adoption.

Chapter 11, Automating Your Data Center, looks at having ServiceNow in charge of your IT. By discovering what is out there, then automating the maintenance and deployment of the new functionality, ServiceNow will become an invaluable part of your business.

What you need for this book

ServiceNow is an enterprise SaaS platform. In order to work with the application, you will need access to a ServiceNow instance, with System Administrator access. The examples in this book are about a custom application, but it is strongly recommended that any work is performed in a subproduction instance.

ServiceNow does offer instances as part of their developer program. You can sign up here: <https://developer.servicenow.com/>. These instances may use a different build and version, which means that some of the examples may not be relevant.

Alternatively, contact your ServiceNow representative and request a temporary sandbox instance.

The examples have all been built with the Eureka version of ServiceNow, released in mid 2015. Both earlier and later versions are likely to work, though some platform and application functionalities will obviously change.

Who this book is for

This book will be most useful to those who understand the basics of IT, web technologies, and computer science. We discuss how ServiceNow implements common design patterns and technologies, enabling you to get a better understanding of how ServiceNow operates.

There is a great deal of functionality in ServiceNow, and it simply isn't possible to cram everything into a single book. ServiceNow has a comprehensive wiki and a popular community forum. The ServiceNow training department has a series of mixed media courses to get you up to speed quickly. We aren't going to replicate these, and the book expects that you have taken the basic admin course as a ServiceNow System Administrator. Two important things to be done are:

- Keep the wiki page open at <https://wiki.servicenow.com/>
- Sign up to the community at <https://community.servicenow.com/>

Every ServiceNow System Administrator needs to have at least a basic understanding of JavaScript. JavaScript underpins the ServiceNow foundations, and, hence, is essential to fully master ServiceNow. An in-depth knowledge of all the intricacies of JavaScript is not needed, since many of the idioms and objects used are ServiceNow-specific. Nonetheless, scripting is part of the product. A review of a good JavaScript manual may be helpful!

We also assume a working knowledge of basic web technologies, such as HTML and CSS, and standard concepts, such as databases and SQL.

Conventions

In this book, you will find a number of text styles that distinguish between different kinds of information. Here are some examples of these styles and an explanation of their meaning.

Code words in text, database table names, folder names, filenames, file extensions, pathnames, dummy URLs, user input, and Twitter handles are shown as follows: "If you remove the `_list` part from the URL, you will be presented with the first record that matches."

A block of code is set as follows:

```
var sa = new SimpleAdd();
sa.increment().increment();
gs.log('Accessing the variable number ' + sa.number)
```

When we wish to draw your attention to a particular part of a code block, the relevant lines or items are set in bold:

```
if (email.subject.indexOf("approve") >= 0)
    current.state = "approved";
if (email.subject.indexOf("reject") >= 0)
    current.state = "rejected";
```

Any command-line input or output is written as follows:

```
ALTER TABLE u_check_in ADD `u_comments` MEDIUMTEXT
```

New terms and **important words** are shown in bold. Words that you see on the screen, for example, in menus or dialog boxes, appear in the text like this: "Create a new field called **Comments**, by double-clicking on **Insert a new row...**"

[ Warnings or important notes appear in a box like this.]

[ Tips and tricks appear like this.]

Reader feedback

Feedback from our readers is always welcome. Let us know what you think about this book – what you liked or disliked. Reader feedback is important for us as it helps us develop titles that you will really get the most out of.

To send us general feedback, simply e-mail feedback@packtpub.com, and mention the book's title in the subject of your message.

If there is a topic that you have expertise in and you are interested in either writing or contributing to a book, see our author guide at www.packtpub.com/authors.

Customer support

Now that you are the proud owner of a Packt book, we have a number of things to help you to get the most from your purchase.

Downloading the example code

You can download the example code files from your account at <http://www.packtpub.com> for all the Packt Publishing books you have purchased. If you purchased this book elsewhere, you can visit <http://www.packtpub.com/support> and register to have the files e-mailed directly to you.

Errata

Although we have taken every care to ensure the accuracy of our content, mistakes do happen. If you find a mistake in one of our books – maybe a mistake in the text or the code – we would be grateful if you could report this to us. By doing so, you can save other readers from frustration and help us improve subsequent versions of this book. If you find any errata, please report them by visiting <http://www.packtpub.com/submit-errata>, selecting your book, clicking on the **Errata Submission Form** link, and entering the details of your errata. Once your errata are verified, your submission will be accepted and the errata will be uploaded to our website or added to any list of existing errata under the Errata section of that title.

To view the previously submitted errata, go to <https://www.packtpub.com/books/content/support> and enter the name of the book in the search field. The required information will appear under the **Errata** section.

Piracy

Piracy of copyrighted material on the Internet is an ongoing problem across all media. At Packt, we take the protection of our copyright and licenses very seriously. If you come across any illegal copies of our works in any form on the Internet, please provide us with the location address or website name immediately so that we can pursue a remedy.

Please contact us at copyright@packtpub.com with a link to the suspected pirated material.

We appreciate your help in protecting our authors and our ability to bring you valuable content.

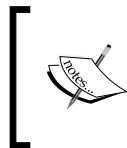
Questions

If you have a problem with any aspect of this book, you can contact us at questions@packtpub.com, and we will do our best to address the problem.

1

ServiceNow Foundations

This opening chapter picks out the most significant foundations of ServiceNow, starting from the bottom up. Understanding the fundamentals of the ServiceNow platform is important. It gives insight into the concepts that underpin how everything else works.



Although long, the chapter is not exhaustive and does expect a basic familiarity with the ServiceNow interface. Remember to reference the ServiceNow documentation and any training material you may have.

Perhaps you've decided to build a new hotel and you want to ensure it won't fall down. The architect's drawings need to be understood and the right building materials ordered. It's costly (and career limiting!) if it collapses in the week after opening!

In this chapter, we review the blueprints. We understand the important design aspects of ServiceNow so that we can build on them later. The data structure available to us is critical, to enable us to model information and processes in the right way.

In this chapter, we will cover:

- The physical components of the ServiceNow architecture
- How everything you see and do is in the database
- A review of the most important field types
- The magic of reference fields and table inheritance
- Using and building a good interface

Diving into the infrastructure

An **instance** is several things. It is a URL (something like `https://<instance>.service-now.com/`); it's software running in the cloud; it's your copy of the ServiceNow platform.

ServiceNow provides a platform and suite of applications as a service. They worry about the hardware, Internet connectivity, and operating system security and provide you with the URL. All you need to get going is a web browser.

Being in charge

An instance is an independent implementation of ServiceNow. It is isolated and autonomous, meaning your instance is not shared with other customers. ServiceNow uses a single-tenancy architecture, which means your instance is yours: you can do what you want with it, such as changing logic, updating the UI, and adding fields.

Every customer has several instances; again, all isolated and independent. One instance might be marked out for developing on, another for testing, and one for production. And, because each instance is independent, each one can be running a different release of ServiceNow. The production instance only differs because it has more powerful hardware.

[ *Chapter 9, Moving Scripts with Clones, Update Sets, and Upgrades,* discusses how you can use your instances for tasks such as building functionality, testing it, and then making it live.]

Changing your instance

A new instance starts with a few ServiceNow applications, some demo configuration, and example data. This is often called the out-of-the-box state. One of the example data elements is the **System Administrator** user. You are able to log in and get going, getting full control immediately.

Everyone makes changes to their instance. Unless the people who will be using the system are called **Beth Anglin** or **David Dan** (some of the default example users), you'll need to load some users at the very least. Some ServiceNow customers configure a lot and some do the bare minimum. You can choose how much you wish to do. Because it is single-tenant, you can alter the configuration and data in almost any way you see fit. Now, it might not always be smart to do that, but you can!



My favorite analogy, if you haven't guessed it, is a building. ServiceNow gives you an office that is yours. It starts off identical, built to the standard plans, but you can redecorate or remodel as you see fit. Perhaps even knock down a wall! (Let's hope it's not load-bearing.) This is the benefit of single-tenancy.

Multitenancy might be an apartment in a block. It is generally more efficient to pack lots of people together in a single building, and you can build it pretty high. However, you don't have the flexibility that being in control gives you. The landlords of the block won't let you knock down a wall!

The vast majority of customers have their instance hosted by ServiceNow. This means the people who built the house will also look after it, on their land. You get great economies of scale, and the benefit of tools and automation design to perform maintenance and give support fast. All the gardeners and janitors are on site, ready to work — they don't need to travel to help out.

Choosing functionality with plugins

All ServiceNow functionalities are delivered as **plugins**. When an instance is turned on, one of its first tasks is to load all the plugins that are turned on out of the box. There are quite a few of those, over 200 in the Eureka version of ServiceNow. And there are several hundred more that you can turn on if you want. A plugin may provide an app, like Human Resources Management, or provide new platform functionality, like Domain Separation. Each new version of ServiceNow brings new plugins and updates to existing ones.



Chapter 9, Moving Scripts with Clones, Update Sets, and Upgrades, talks about plugins and upgrading ServiceNow in more detail.

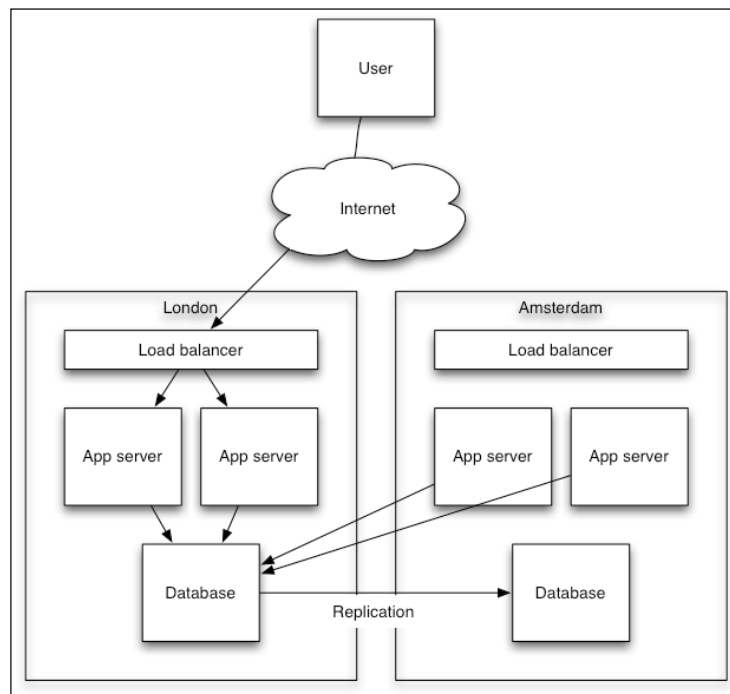
When a plugin is turned on, all the data and configuration that the application needs is loaded into the database, meaning that it is ready for work in just a few moments. Many also contain demo data, giving you examples of how it could work.

Digging into hosting

A ServiceNow-hosted instance is split over two physical datacenters, a high-availability pair. Each location runs independently of the other, giving a semiclustered environment. In the event of a catastrophic disaster, with one location being completely unavailable, the other nodes will just pick up the load, with almost no downtime. In fact, the process of switching between locations is used for maintenance procedures, enabling your instance to be well-protected against hardware and other failures.

When you visit your instance, you are directed through several layers:

- By looking up the DNS records, you are directed to the currently active datacenter
- The load balancer, by reading a cookie, directs you to the application server you have your session with
- If you aren't logged in, you get directed to the least-busy application server
- Your application server then uses the database currently determined as active



Knowing the nodes

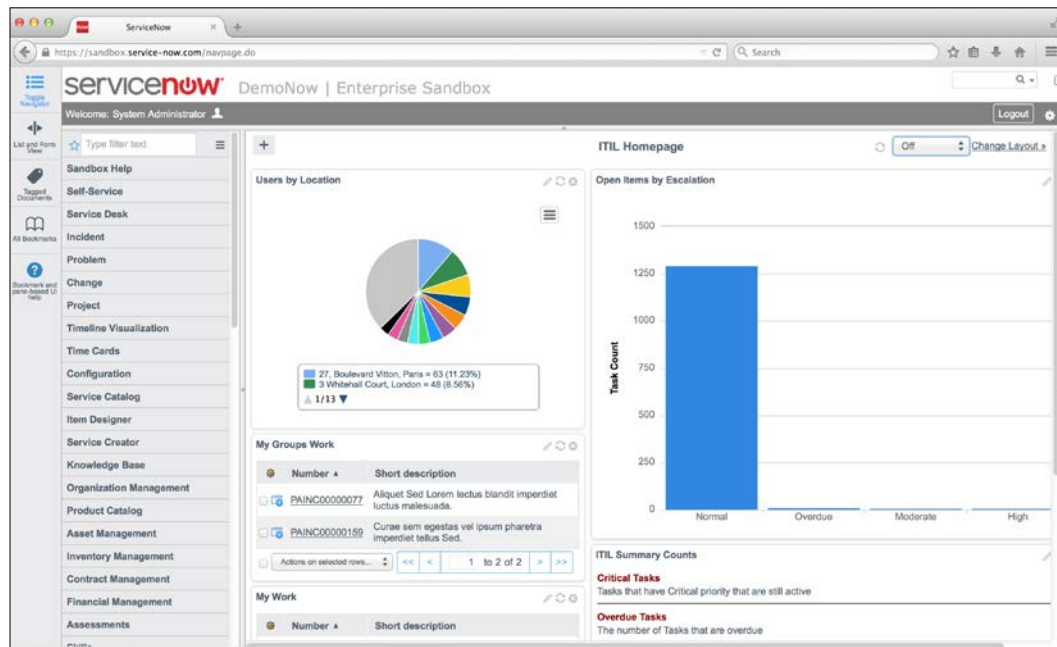
From an architecture perspective, a ServiceNow instance is made up of several application and database servers or nodes. These are generally running on shared hardware, meaning that although your instance is logically separate and independent, it is physically hosted alongside another customer. At each location, there are generally at least two application nodes, each running a copy of the ServiceNow platform, which work together to share load. Additionally, there may be worker nodes installed to process the non-interactive jobs, such as event processing. Even though you'll never directly log in to these worker nodes, they perform some background processing, allowing the interactive application servers to respond more quickly to user requests. For example, a worker instance might send out e-mails or deal with integrations. While there are generally lots of application nodes, there is only one active database server, running on a separate physical server. It does have a redundant pair hosted in the remote datacenter.

[ *Chapter 5, Events, Notifications, and Reporting*, explores the concept of event queues in more detail.]

Exploring the database

So you've got an instance and have logged in. Great! What can we see? We can see database records.

You may not realize it, but the homepage, the reports, and the menus to the left are all database records:



Almost everything in ServiceNow is an entry in a database. When you look at the user interface, virtually everything you see—from the data typed in by a user, to log files, to how the views are structured—is stored in the instance's relational database. Even the scripts you write are kept in a string field in a record, and the files you upload are stored in chunks in the database.

Everything is built on the top of this structure. You don't need to reboot the server to apply new functionality; you are just updating data records. You don't need to reload configuration files—any properties you set will be read on the next operation. Even the database metadata, information about the fields themselves, is stored in another table.

This gives you extraordinary control and ability. You can organize, search, and manage the data in an unprecedented manner. You can find scripts the same way you find users, by searching tables. You can control and secure any data, regardless of what it is, by using Access Control Rules. This means you can focus on designing and building great business applications, since the platform works in a consistent manner.



ServiceNow may be considered a high-level platform that is based on the concept of *Model-View-Controller*. When building a ServiceNow application, you can first think of the data structure. You determine what information you need to store and how it all links together, creating tables and fields. This is the *model* aspect. Automatically, you get a simple *view* on this data, with forms and lists showing your information. And you can build simple ways to manipulate and change the data, through automation and simple manual updates. This is the *controller* aspect.

Introducing the Gardiner Hotel data structure

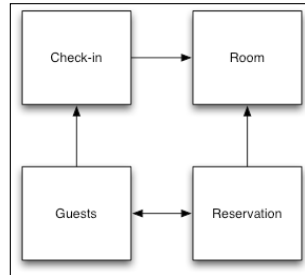
One of the first things that many people learn how to do in ServiceNow is to add a field. In ServiceNow, this is a straightforward operation – you add the new field to a form or a list using the UI. Under the covers, the ServiceNow platform is performing, among other operations, a simple SQL command to add a column to the table you are manipulating. When you add a field, you add a column to the table. When you remove a field, you are dropping it. There is no magic – the platform interface just makes it easy to do.



ServiceNow allows you to add fields to every table in the system. If you decide that adding another is a good idea, you have the power to do it!

In order to work through the ServiceNow functionality, we are building a hotel management application for your new employer, Gardiner Hotels. It will involve building a simple data structure, but one that is highly interlinked.

Here is a representation of the tables we will create in this chapter:



This diagram represents several common activities within a hotel, and their links to each other.

- **Guests:** The reason why Gardiner Hotels exists! Our guests' details are the most important information we have. We definitely need their names, and optionally their e-mail addresses and phone numbers.
- **Room:** This represents where our guests will sleep. We store the room number and the floor it is on.
- **Check-in:** When guests want their room key, they check in. We record who is checking in to a room, when, and who made the entry.

We will create a link to the **Room** table so we can easily see information about the room, such as what floor it is on.

- **Reservation:** Our guests like staying with Gardiner Hotels, and they often book months in advance.

One reservation might be for many guests, especially if the whole family is coming. A big family might need a big room. We need to record where exactly they may stay.



Over the course of the book, we will expand and further develop the application. Its primary use is to show you as much of the capability of ServiceNow as possible, so some of the examples may be done better in other ways.

Creating tables

Firstly, let's create an app to hold our configuration in. Navigate to **System Applications > Create Application**. Fill in the following value and click on **Submit**:

- **Name:** Hotel



The convention for navigating through ServiceNow uses the following structure: **Application Menu > Module**. For modules with separators, it will be **Application Menu > Section > Module**. The easiest way to find a particular link is to type it in to the **Application Filter** at the top left of the menu. Make sure you are choosing the right one, though, because some modules are identically named.

Then find the **Tables Related List**, click on **New**, and fill in the following values:

- **Label:** Check-in
- **Name:** u_check_in

Table [Submit] [Cancel] [Help] [Info]

A table is a collection of records in the database. Each record corresponds to a row in a table, and each field on a record corresponds to a column on that table. Applications use tables and records to manage data and processes.

[More Info](#)

* Label Create access controls ☒

* Name * User role [Search] [Add]

Extensible ☐ Live feed ☐

Extends table [Search] Auto-number ☐

Create module ☒

Add module to menu

Table Columns Search [Search] [Add]

Dictionary Entries

Column label	Type	Reference	Max length	Default value	Display
Insert a new row...					

[Submit] [Cancel]

The rest of the default entries should be fine.



Many technical items in ServiceNow have a label and a database name. In this case, the database table is called `u_check_in`, while an entry in the **Field Labels** [`sys_documentation`] table contains the mapping between the name and the label. I'll use the **Label** [`database_name`] format throughout the book.

When you click on the **Save** button, the application server creates the table in the database.



I recommend using the **Save** button to commit records to the database, accessible via the 3-line menu icon, or by pressing `Ctrl + S`. This ensures that the record is renamed on screen rather than redirecting to the previous page.

Adding fields

When you create a new table, you get some system fields. They include two dates (when a record was created and when it was last updated), two string fields (containing the user ID of who created it and who updated it), a unique GUID called the `sys_id`, and a field that counts the number of updates to the record. They are all updated automatically, and it is generally good practice to leave them alone. They are useful just as they are!



We'll be discussing the `sys_id` field a lot in just a moment!

The following screenshot shows how the system fields are represented within the tables:

Table Columns Search for text						
Dictionary Entries						
	Column label	Type	Reference	Max length	Default value	Display
	Created by	String		40		false
	Created	Date/Time		40		false
	Sys ID	Sys ID		32		false
	Updates	Integer		40		false
	Updated by	String		40		false
	Updated	Date/Time		40		false
Insert a new row...						

 The autogenerated fields are very helpful to the System Administrator for finding records quickly. I always add the **Updated on** field to my lists, since it makes finding the records I've been working on (such as scripts) much faster.

In addition to these automatic fields, we need to create some of our own. We will need several, but right now, let's create something to store any requests that the guest may have. Perhaps they may have specifically requested a high floor.

Create a new field called **Comments** by double-clicking on **Insert a new row** Fill out the row with the following data and then save the record:

- **Column label:** `Comments`
- **Type:** `String`
- **Max length:** `500`

The following screenshot displays how the columns list looks before saving the changes:

Table Columns Search for text						
<< < 1 to 6 of 6 > >>						
Dictionary Entries						
	Column label	Type	Reference	Max length	Default value	Display
✗	Created by	String		40		false
✗	Created	Date/Time		40		false
✗	Sys ID	Sys ID		32		false
✗	Updates	Integer		40		false
✗	Updated by	String		40		false
✗	Updated	Date/Time		40		false
✗	Comments	String		500		false
✎ Insert a new row...						

 All the fields and tables that you create will get a `u_` prefix in the name, while the label will stay as is. This helps you (and the upgrade process) to separate them from the out-of-the-box functionality.

Upgrades are discussed in more depth in *Chapter 9, Moving Scripts with Clones, Update Sets, and Upgrades*.

Knowing what's happening

Behind the scenes, the application server is running SQL commands against the database. Specifically, at the time of creating the field, the following was executed:

```
ALTER TABLE u_check_in ADD 'u_comments' MEDIUMTEXT
```



If you wish to see these commands, navigate to **System Diagnostics > Debug SQL**. This will place lots of information at the bottom of the page. Other diagnostic tools like this are discussed in *Chapter 8, Diagnosing ServiceNow – Knowing What Is Going On*.

This demonstrates a key concept. Whenever you perform an action in ServiceNow, it results in a string of database commands. The database is altered and the platform's internal state is updated. These actions are generally carried out quickly, with the whole process completed in about half a second. No downtime is necessary.

Introducing the dictionary

The **dictionary** is a metatable. It describes your table and fields – what their name is, how big they are, along with any special attributes they may have. For example, one field might be dependent upon another. A dictionary entry might also be referred to as an **Element Descriptor**.

The table is represented by an entry in the dictionary with a table value, type of **Collection**, and no column name. When the **Comments** field was added to the **Check-in** table, the platform also made a dictionary entry with the table and a column name. You can view it by navigating to **System Definition > Dictionary**.

The options that are available in the dictionary are dependent upon what type of field you are working with. Reference fields, which we will explore in a moment, have extra functionality that is controlled here, such as dynamic creation.

As we work through ServiceNow, we'll spot functionality that is enabled through the dictionary. However, much of it can be achieved in other ways, often in a better manner.



Older versions of ServiceNow have a read-only tick box available on the dictionary form by default. While it can be included and used in later versions, it is usually a better idea to use security rules instead. Having a field marked as read-only is a binary choice, and giving System Administrators control over data is a good idea. You get that granularity with security rules, which we will explore in *Chapter 7, Securing Applications and Data*.

The easiest way to navigate to a dictionary entry is by right-clicking on the label of the field and choosing **Personalize Dictionary**. You can see some details about a field, such as what table is it in, by choosing the **Show** option on the menu that appears on right-clicking.

The Globally Unique Identifier

The ServiceNow database is a relational database. This means that one bit of data can relate to another. To ensure that every record or row can be referenced easily, every record has a unique identifier: a **primary key**.

In ServiceNow, this primary key is something that isn't related to the data itself. It is a **globally unique identifier** or **GUID**. This GUID is stored as a 32-character string, made of hexadecimal characters (the numbers 0-9, plus the letters a-f). The number of unique GUID values is so large that the probability of two accidentally being the same is negligible. This is an example GUID: 5137153cc611227c000bbd1bd8cd2005.



This type of identifier is sometimes known as an **OID** or **object identifier**. It has no special significance; it just uniquely identifies a data row. It can also be called a **surrogate key**.

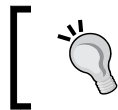
Whenever you create a record in ServiceNow, the platform generates a new GUID. The characters that are generated are random—a mixture of several sources, including the date and time and details specific to the instance, meaning it is not sequential or predictable. The GUID is saved alongside the record, in a special field called `sys_id`. The `sys_id` field is heavily used in the ServiceNow platform—you will start seeing GUIDs everywhere!

As an example of how ServiceNow uses the `sys_id` field, conduct the following experiment. Construct a URL similar to the one that follows, substituting `<instance>` with the name of your instance, and visit it with your browser:
`https://<instance>.service-now.com/sys_user.do?sys_id=5137153cc611227c000bbd1bd8cd2005.`

In a new instance, you should happen across the user record of **Fred Luddy**. (If the demo data has been removed, you will get a **Record not found** message.)

It is useful to examine the structure of the URL. Firstly, spot the `sys_user` table. Then spot the GUID. With these two items, the instance knows exactly what data it needs to pull up and present to you.

Every record has a `sys_id` field. It is the only field that ServiceNow really cares about. It also looks after itself. You don't need to worry about it during day-to-day operations.



Reference fields, as we'll see, are very reliant upon the `sys_id` fields. When we get into scripting in *Chapter 2, Server-side Control*, you'll be seeing more of them!

Every other field is non-mandatory and non-unique to the database platform. You can have two records that are otherwise identical but only have a differing `sys_id` field. (It is possible to enforce uniqueness in other fields too, as we'll see later.)

This means that, in general, you can change the value of fields to whatever you like and still maintain referential integrity; no system errors will occur. If you want to rename a user or a group, go ahead. Since everything related to that user will be associated to it via the `sys_id` field, the name of the user is not important.

Many other products do not use surrogate keys; data is linked together using user-provided data. If you change the name of a group, for example, this can remove all group memberships and assignment to tasks. Not in ServiceNow!

 An important exception to this behavior is with roles. Roles are referred to by their name in scripts, so if you change the name, all scripts that use it will need to be altered (though security rules *do* refer to the role through the `sys_id` field.) In general, it is a good idea to keep the name of roles the same.

It is a good idea not to interfere with this flexibility. When you are building functionality, try not to refer to records using their name or `sys_id` in scripts. Instead, use the properties or attributes of the record itself to identify it. So, rather than hardcoding that a particular room in our hotel needs special treatment, create another field and use it as a flag. The VIP flag on the **User** table is a good example of this.

Building hierarchical tables

ServiceNow is built on a relational database. Instances hosted by ServiceNow use MySQL—a popular open source database that is robust, well featured, and scalable. Relational databases are relatively simple to understand, which is one of the reasons why they are most commonly used; data is held in a tabular format with each table storing information about a particular item. Relationships may exist between these items. Our database design described in the previous section has four tables, each relating to the others. For example, the **Check-in** table will relate to the **Guest** table to know who checked in.

 The ServiceNow platform can run on almost any relational database, such as Oracle or SQL Server. But supporting different architectures is difficult, so it is not a standard offering.

Benefiting from an object-oriented design

The simplicity of a relational database means that, on its own, it does not easily represent the data structures used in modern object-oriented programming languages. One particularly useful function of an object-oriented approach is inheritance.

 Inheritance allows one object to build on the functionality of another. Why duplicate effort when you can reuse existing capability?

In ServiceNow, a table can inherit another. The parent table defines the base functionality, while the child table, built on top, can continue to use it. That means that any fields you add to the base table are automatically available to the child too. In fact, almost all functionality you add to the base table is available to the child.

In our **Hotel** application, we want to store information about our guests. We need to know their names, their telephone numbers, and perhaps their addresses. ServiceNow has got a built-in table for storing people—the **User** table. But we want a special type of person—guests. Let's keep staff in the **User** table and guests in a new extension table.

 The **User** table in ServiceNow defines who can log in and use the platform functionality. Sometimes you need a contact database, which stores information about people: their names, phone numbers, location, and who their manager might be. It's tempting to build the contact database as a separate table and keep the two separate, but I recommend using the **User** table as the basis for both. It saves the duplication of data and allows reuse of a special functionality that is built specifically for the built-in table. But do be aware of licensing.

Extending the User table

Let's extend the ServiceNow **User** table in order to have a special class for guests. First, we have to mark the **User** table as extendable. Go to **System Definition > Tables** and find the **User** [sys_user] table. Make the following change and save once you're done:

- **Extensible:** <ticked>

 It is likely you will get a pop-up message asking if you want the **User** table to be included in the **Hotel** application. Click on **Leave Alone**; we want to keep things as they are. We don't want to include the **User** table in our application, only the **Guest** table.

Now, go to our application by navigating to **System Definition > Custom Applications > Hotel** and create another new table. Use the following data:

- **Label:** Guest
- **Name:** u_guest
- **Extends table:** User

If you look at the fields available on this new table, you'll see lots of fields already, beyond the normal automatic five. These additional fields are those defined on the **User** table.

 	Column label	Column name	Type	Reference	Max length
 	Accumulated roles	accumulated_roles	String		4,000
 	Active	active	True/False		40
 	Building	building	Reference	Building	32
 	Calendar integration	calendar_integration	Integer		40
 	City	city	String		40
 	Company	company	Reference	Company	32
 	Cost center	cost_center	Reference	Cost Center	32
 	Country code	country	String		3

What does this mean? The **Guest** table has *inherited* the fields of the **User** table. This is extremely useful; I don't need to create the name, telephone, and e-mail fields – they are already available for me.

Indeed, when you create a table that inherits another, you gain all the functionality of the parent. Most of the scripts, rules, and policies of the parent automatically apply to the new table. But sometimes you want to create the functionality for the child table only. So, ServiceNow lets you place it at the level you need.



We'll cover how scripts are handled in ServiceNow in the next chapter.

Interacting with hierarchical tables

Our new table is the right place for storing information about our valued customers. While useful fields have been inherited from the **User** table, it doesn't contain everything. Make a new field to store their membership number. For simplicity, create a text field with a Column label named `Membership Number`.

To do this, click on **Design Form** on the **Guest** table record. Find our new field from the left-hand-side list, drag it underneath the **Last name** field in the layout, and click on **Save**:

[ Check out the wiki if you need help on this step:
http://wiki.servicenow.com/?title=Form_Design.]

To test, let's create a new guest record. Navigate to **Hotel > Guest** and use this data:

- **First name:** Alice
- **Last name:** Richards
- **Membership number:** S2E1

Great! We can enter a membership number fine. And if we look at a standard **User** record such as **Fred Luddy** (**User Administration > Users**), the **Membership number** field does not show up.

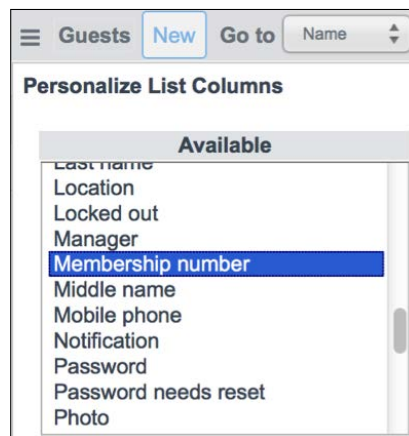
Viewing hierarchical tables

You may have noticed that our new guest, Alice, showed up when you visited the **User** table. That's because Alice is both a user and a guest. A **Guest** record will be treated just like a **User** record, unless there is something specific that overrides that behavior. In our case, the only difference between a **User** and **Guest** right now is that the latter has an extra field.

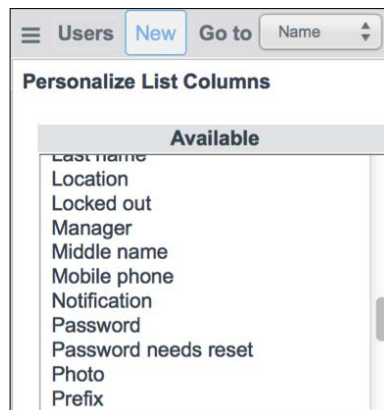


This behavior is called polymorphism, and is a very useful functionality. Use the base or extended functionality, as you need it.

But this gives rise to something that confuses many. If I look at the **Guest** table, I can add, through **Personalize List**, the **Membership number** field.



However, if I try to add a **Membership number** on the **User** table, I can't.



This is because a **User** doesn't have a membership number; only a **Guest** does. Think carefully about where you position fields to ensure they can be seen at the right level.

 You can turn on a property that lets you view inherited fields in the base table. This is discussed more in the *Dot-walking* section in this chapter, as well as the *Reporting* section, in *Chapter 5, Events, Notifications, and Reporting*.

Overriding field properties

Inherited fields allow you to easily reuse functionality. However, sometimes, you want the fields in the extended table to work differently to the base table. This is accomplished with dictionary overrides.

For example, let's change the default time zone for new guests so that it's different to that of the **User** table. The current default for **Users** is the system time zone, and **Guests** inherits this setting.

Navigate to the dictionary entry for the **Time zone** field (perhaps through **System Definition > Dictionary** or by right-clicking on the label) and look for the **Dictionary Overrides** tab. Click on **New**. Use the following data:

- **Override default value:** <ticked> (tick this option)
- **Default value:** Europe/London (or your own choice!)

Now, when you create a new **Guest** record, it sets the default time zone to be Europe/London. Any new **User** records will be unaffected.

 You can also change field labels so that they are different for the base and extended tables. Navigate to **System Definition > Language File** and create a new entry, populating **Table** with the extended table name (such as `u_guest`). The **Element** field should be the field name.

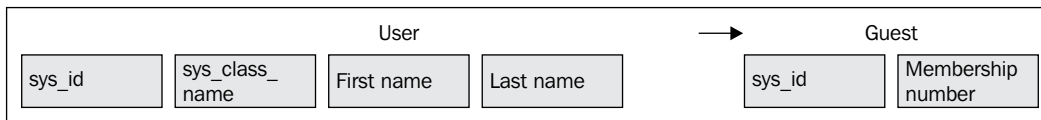
Understanding the background behavior

You might be wondering how this all works.

A child table is a normal database table. However, it does not recreate all the fields of the parent. Instead, the only columns in that new table are the new fields. For example, if I were to run the `DESCRIBE u_guest` SQL command on the database, I'd only see two fields: `u_membership_number` and `sys_id`.

So, when I look at Alice's record on the **Guest** table, the ServiceNow platform is actually joining the parent table and the child table together, behind the scenes. ServiceNow takes the independent tables and (invisibly) joins them together, creating the illusion of a single, bigger table.

Our friend, the `sys_id` field enables the platform to do this. If you remember, the `sys_id` field uniquely identifies a record. In the case of an extended table, the `sys_id` field is actually stored in two places: on both the parent and child tables. The platform joins both together whenever you query the **Guest** table.

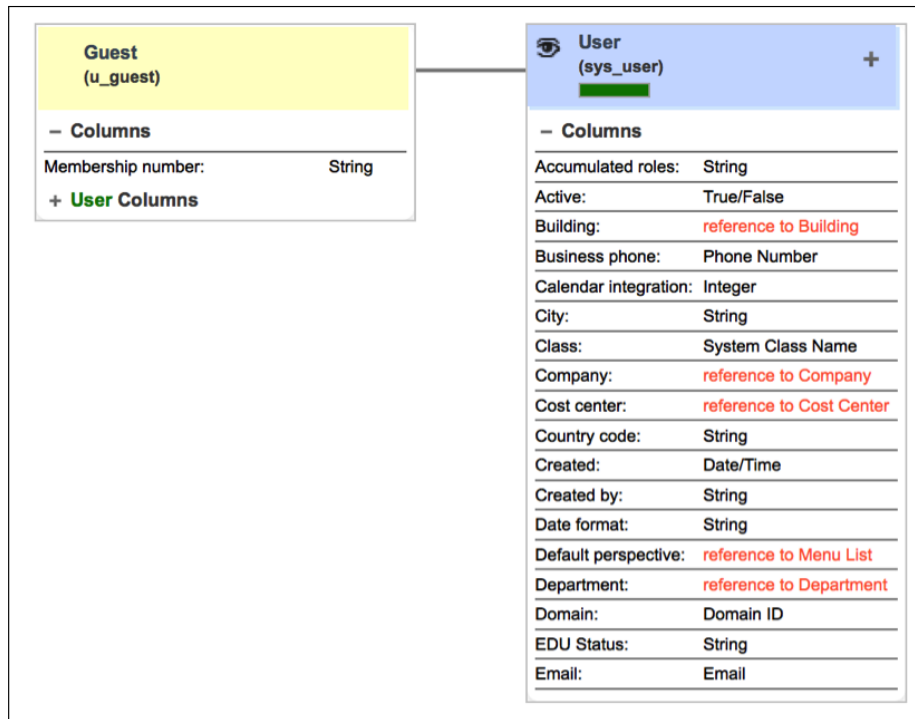


When you mark a table as extendable, you are also adding a second system field: **Class** [`sys_class_name`]. It contains the name of the table that the record represents. For example, the **User** record representing **Fred** would have `sys_user` in the **Class** field, while the **User** record for **Alice** would be `u_guest`. With this information, ServiceNow can join tables if necessary and present you the appropriate data.



In the Dublin version of ServiceNow, this backend behavior has changed. There are now two models for table extension: hierarchical and flat. The hierarchical method consists of multiple tables that are joined together as needed just as described, while a flat structure consists of one very large table with all the columns of every table. When you make a new table and add a new field, in reality it is simply adding another column to the base table. The platform again hides this from the user. This does not have an impact on how table extension works in ServiceNow and is purely undertaken for performance reasons on larger instances.

The ServiceNow interface knows about this behavior. When you navigate to a record, ServiceNow will always show you its actual class. So, even if I am viewing a list of **Users**, when I click on **Alice**, I will see the **Guest** form, with all of the appropriate attributes. The Schema Map is really helpful to visualize what is going on. Navigate to **System Definition > Tables**, choose the table you are interested in, and then click on **Show Schema Map**. The following screenshot shows the Schema Map of the **Guest** table.



This wiki article has more information on the Schema Map: http://wiki.servicenow.com/?title=Schema_Map_for_Tables.

Changing class

Once a record is stored in a particular table, a neat trick is to move it. If I decide that Alice is actually a user, I can alter the value of the **Class** field. The platform will drop any **Guest**-specific information and start working just like a **User**. The **Class** field can be added to a form or list, and is represented as a choice list. Often, you will want it to be read-only.



The ability to change the class is a powerful feature and you should be aware of the consequences. It is unusual to reclassify a record, and it may throw off reporting; for example, if you counted ten users and nine guests, and suddenly one switched, you might have an overbooking. It may also clear reference fields, which we are about to discuss.

So far, we've discussed how you can add fields into a specific class, and seen how they are inherited. But this will work with much more than fields! As we work through the chapters, we'll see how functionality such as Business Rules, Access Control Rules, and Import Sets all benefit from hierarchical tables.

Storing data

There are lots of different fields provided by the ServiceNow platform. We will explore some of the simpler ones first, before moving on to the fundamental backbone of the data structure with reference fields:

- **Journal input:** These fields provide an always-empty textbox designed to store notes, but does not store the information in the field on which it's created. Instead, when you enter text into a journal input field, it's saved in the **Journal Entries** [sys_journal_field] table.
- **Journal list:** They don't have any entry capabilities at all. Instead, you make it dependent upon a journal input field, and it shows all its entries by querying the **Journal Entries** table automatically. This allows you to have two journal input fields for one journal list. Journal lists are also known as Journal outputs.
- **Journal:** These are a combination of both Journal input and Journal lists, doing both jobs together.
- **Choice:** They are text fields, but rendered as HTML select fields. The value that is stored in the database is plain text. Another table, the **Choices** [sys_choice] table, stores the options and labels. This lets the platform convert "wip" in the database to present "**Work in Progress**" to the user. Any values that don't have a label are highlighted in blue in the dropdown.
- **Integer choice:** These fields use numbers instead of text to achieve the same result. They are useful for representing states, since they allow you to use greater than or less than conditions, but have proven difficult to work with since the numbers don't mean much!



Use caution when dealing with the out-of-the-box integer choice fields, such as **State** on the **Task** table. If you reuse them (which is a good idea), you should always align your states to the existing ones. For example, **3** should represent **Closed**. If you do not align them, then users will be confused when reporting. This is discussed in detail in *Chapter 4, Getting Things Done with Tasks*.

- **Currency:** They are string fields that combine the currency and the amount together. `USD;1000` represents \$1,000. The platform uses this information to provide conversions between different currencies. For example, if I prefer to see amounts in GBP, the platform will, if it has the latest currency rates, display £675.
- **Date:** There are several **Date** fields in ServiceNow. The time is stored as UTC in the database, and the appropriate display value is calculated by the user's profile.
- **True/False:** These fields are simple Boolean values in the database. They are rendered as tick boxes.
- **URL:** These fields provide a space to enter a link, which can easily be made clickable.
- **HTML and Wikitext:** Other fields, like these, provide different interfaces to manipulate strings. It is tempting to use HTML fields in lots of places, but they do come with overhead, and browsers have different capabilities. For example, Firefox is able to encode an image into a data URI and store it in these fields, while IE is not able to. Test carefully if you want to use capabilities like this.

Attachments

In addition to text, ServiceNow can also store binary data. This means that anything (images, music, or even a multitude of PowerPoint documents), can be saved in ServiceNow. Just like everything else, binary data is stored in the database. However, rather than using a blob field, binary data is split into 4k chunks and saved into the **Attachment Documents** [`sys_attachment_doc`] table. Each chunk of a file refers back to the **Attachments** [`sys_attachment`] table, where the filename, content type and size, and other metadata are stored.

An attachment is always related to another record. Information on this other record is stored with the other metadata in the **Attachments** table. For example, if a **Reservation** record had a PDF of the booking form attached to it, the **Attachment** record would contain the file name of the document, as well as the `sys_id` of the **Reservation** record.



We'll see in later chapters that there are often better ways than manually adding attachments containing booking information. Why not have the e-mail come directly into ServiceNow? (We'll see how in *Chapter 5, Events, Notifications, and Reporting*.) Or even better, have the guests perform the booking directly with ServiceNow? (*Chapter 10, Making ServiceNow Beautiful with CMS and Jelly*, shows us how to do this.)

Setting properties

One of the simplest ways to control the platform is to set properties. There are lots of things you can change by just clicking on a box or changing a value. And just like everything else in ServiceNow, the configuration properties that you set are stored in a table: the **System Properties** [`sys_properties`] table to be precise.

To see how many options you can choose, type in `Properties` into the filter text of the **Application Navigator**. The many matches are shown, including **System Properties > UI Properties**. This collection contains some very useful options, including how forms look and feel, if list editing is enabled, and whether **Insert and Stay** is always available. You may want to take some time to find out what they do.

Some properties are not categorized, but all are accessible via **System Properties > All**. This gives a large list – almost 700 in Eureka. This book will guide you to the more relevant ones, but many are documented on the wiki:

http://wiki.servicenow.com/?title=Available_System_Properties



In older versions of the platform, the list is available by typing in `sys_properties.list` in the filter text of the **Application Navigator**.

Reference fields

When designing the data structure for a hotel, you may want to link a guest's record with the room they have checked in to. It won't be good for business if we don't know who has checked in where! This is exactly what a reference field does; it creates a link between two records, one pointing to another.

When we examined the URL for a particular record, it contained two parts: the table and the `sys_id` value of the record. These are the two items needed to reference a record. So when you create a reference field, you need to select which table it should point to. And the contents of the field will be a 32-character string. Sounds familiar? Yep, you will be storing a `sys_id` in that field.



Reference fields are one of the most important items to understand in ServiceNow. The database sees a string field containing the `sys_id`, a foreign key. However, this is meaningless to a person. Therefore, the platform allows you to pick a field that will be displayed. For a person, this might be their name. Other records might have a user-friendly reference number. This is usually an incremental number; there are scripts that can generate one automatically. You can choose which field to show by ticking the **Display** field in the **Dictionary** entry, but this is only useful to the user. Only the `sys_id` is important to the platform.

Reference fields are used throughout ServiceNow, just like a proper relational system should. Scripting, lists, and forms all understand references fields, as we'll see as we work through the chapters.

Creating a reference field

Let's think about something that the **Hotel** application needs—a room directory. Each room has several attributes that defines it: its room number, how many beds it has, and what floor it is on. We can represent this information as fields in a **Room** record, all stored in a dedicated room table. Once we have this, we can modify the **Check-in** table to record which room has been taken by which guest.

As before, navigate to the **Hotel** record under **Custom Application**, and create a new table with the following data:

- **Label:** Room
- **Name:** `u_room`

Create a field to store the room number:

- **Column label:** Number



Don't use the **Auto-number** option, but create a new field using the **Related List**.

And another field to store the floor it is located on:

- **Column label:** Floor
- **Type:** Integer

The final result should look like this:

Table - Room [Update] [Delete] [Delete All Records]

* Label: Room

* Name: u_room

Extensible: ☐

Create access controls: ☒

* User role: hotel_user

Live feed: ☐

Auto-number: ☐

Table Columns Search: for text

Dictionary Entries

	Column label	Type	Reference	Max length	Default value	Display
	Number	String		40		false
	Floor	Integer		40		false
	Updated	Date/Time		40		false
	Updated by	String		40		false
	Updates	Integer		40		false
	Sys ID	Sys ID		32		false
	Created	Date/Time		40		false
	Created by	String		40		false

Insert a new row...

Create a few example records, giving each one a different number. Make several on the same floor. Perhaps one of them might be Room 101 on the first floor.

Rooms [New] Go to: Number

1 to 5 of 5

All

	Number	Floor
	101	1
	102	1
	103	1
	201	2
	202	2

Note that the platform does not force you to choose different numbers for each record. Unless you mark a field as unique, or you create a rule to check, the platform will allow you to create them.



To mark a field as unique, you can edit the dictionary entry of that field. (You will need to configure the dictionary form and add the **Unique** checkbox.) By ticking that field, you are asking the database to enforce it. It does this by making that field a unique key. This has two impacts. Positively, it creates an index on that field. However, if a user attempts to save a duplicate value, they will get a message saying **Unique Key violation detected by database**. This can be a little jarring for a non-technical user. Try to catch the error with a Business Rule first.

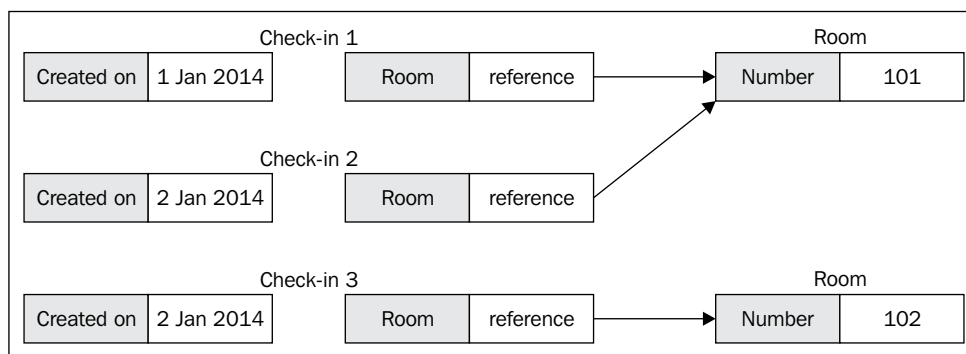
Now that we have a list of rooms, we need to create the link between the **Room** and **Check-in** records.

A reference field can be referred to as a "one-to-many" relationship. Since a room may be checked in to multiple times (one day you have one particular guest, and the next day another may sleep in it after our fabulous cleaners have done their work), but for a single check-in, you can only select one room. You can only sleep in one bed at once!



A classic example of a one-to-many relationship is between a mother and her children. A child can only have one biological mother, but a mother can have many children.

The following diagram shows the relationship between **Room** and the **Check-in** record:



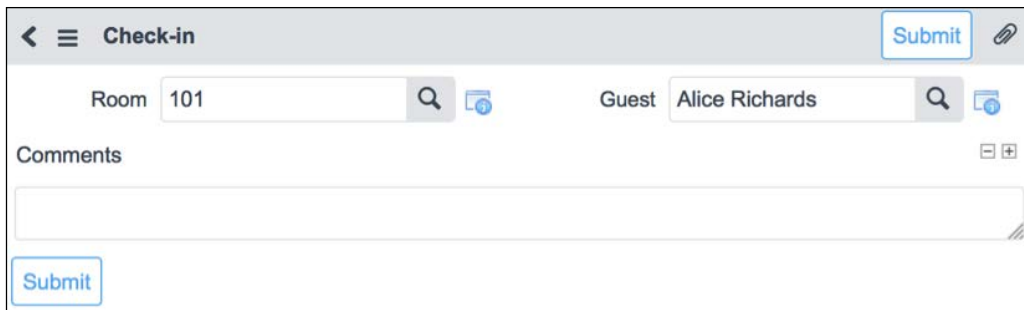
In the **Check-in** table, create two new fields. One for the room, using the following data:

- **Column label:** Room
- **Type:** Reference
- **Reference:** Room

And another for the guest:

- **Column label:** Guest
- **Type:** Reference
- **Reference:** Guest

Now, create a few example **Check-in** records. To simulate someone going into Room 101, create a new entry in the **Check-in** table. You may want to rearrange the form through **Form Design** to make it look a little better.



 Note that when you try to select a guest, only **Alice** is available. Since the reference field is pointing to the **Guest** table, users are not shown.

You can view the **Room-to-Check-in** relationship from both directions. If you are on the **Check-in** form, you can see which room is in use through the reference field. The reference icon is very useful for viewing more details about the record—just hover over it.

 If you hold the *Shift* key on your keyboard while you move your mouse cursor over the reference icon and then into the pop-up window, the information will remain until you click on the Close button. This is quite useful when copying data from that record without losing place.